

**MULTI-PARADIGM MACHINE-LEARNING-AIDED
CONCRETE MIX DESIGN OPTIMIZATION**

DATASETS: CONCRETE COMPRESSIVE STRENGTH, ACI BASED MIX

TEAM CONCRETE CONCLUSIONS

TEAM MEMBERS: GRAEME MARSHALL, LOGAN GRIFFIN

AUGUST 12, 2026

FINAL PROJECT

ABSTRACT

In this experiment, machine learning models are evaluated in order to predict concrete compressive strength and optimize concrete mixture performance. Six regression models are trained and compared using an initial dataset of 425 samples. The best performing model is then used to estimate additional strength values, broadening the dataset to 10,000 points. A generative adversarial network is used to create 40,000 more samples, resulting in a 50,000 point dataset. The best performing model is retrained, feature combinations are evaluated, and the final model is used to predict a mixture that optimizes concrete compressive strength-to-weight ratio, subject to constraints.

INTRODUCTION

The ultimate goal of this experiment is to find a concrete mix with the maximum ratio of 28-Day Compressive Strength to Mix Weight. This 28 day threshold is a critical one, representing a standard benchmark for acceptance. It is used as the full, final strength of the concrete under standard conditions (American Concrete Institute). However, there is a very limited set of data available to address this problem. The “concrete_compressive_strength.csv” data set was the initial basis of this experiment.

This dataset was provided directly by Prof. Rafiei. However, it is also accessible through the UC Irvine Machine Learning Repository and is based on an earlier piece of published research (Yeh, 1998). This data set consists of samples of concrete of varying composition that have been tested for their compressive strength, with numeric values given for this parameter. The samples varied in their concentrations of cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate, and fine aggregate. The samples also varied in age. Of particular interest for this experiment are the only 425 samples which have an age of 28 days.

In order to overcome this relatively small amount of data fit for supervised techniques, an initial regressor study was done to identify a model type that would best serve as a regressor for the remainder of the work. This model type was then trained on all 425 samples from the initial dataset. Then, another 9,575 samples from the “aci_based_mix.csv” data set (also provided by Prof. Rafiei) were acquired. These samples had no output feature data (compressive strength), so the regressor was used to generate estimations which were assumed to be essentially the real values. Then, these 10,000 data points were used to train a Generative Adversarial Network, which itself was used to generate another 40,000 input samples. The regressor was once again used to generate the output feature estimations for these samples. This new “super-boosted” dataset now was over 10 times the size of the limited initial data set of 425 samples, allowing for the subsequent steps to be executed.

A combinatory investigation was performed to identify the best feature combination, using these 50,000 samples. Finally, a new regressor was trained using all 50,000 samples from the now “super-boosted” dataset, and using only the feature combination identified. This regressor was then used in the objective function of an optimization study on the strength to weight ratio as constrained by limits on total mix weight and water to cement ratio. Further details of the

models and methods used follow, as well as the results of the various stages and final conclusions.

MODEL/METHOD

Identifying Regressor

The first phase of the experiment involved identifying a proper regressor for this data. Six Scikit-Learn models were chosen for evaluation: Linear Regression (LNR), Support Vector Machines (SVR), Decision Trees (DTR), Random Forests (RFR), Gradient Boosting (GDB), and K-Nearest Neighbors (KNN). Two RTTs (0.05 and 0.1) and 10 RRSs were executed for each, leading to a total of $2 \times 6 \times 10$, or 120, models trained on the initial dataset of 425 samples. These models were then evaluated based on their mean squared errors, and GDB was chosen as the best model to move forward with for further work. A brief summary of each of the evaluated model types follows in the models subsection below.

A GDB model was then trained with all 425 of the original data points. The scaling for the input and output domains of this model was saved for later use. This model is referred to as “Regressor 1.”

Models:

- LNR: works by minimizing the sum of squared differences between the known outputs in a dataset and the outputs predicted by the model in order to find a best fit relationship for estimating the regression output.
- SVM: learns how to estimate values by finding a hyperplane that fits closely to the data points. An epsilon value is used to represent a radius around the hyperplane. The data points lying on the radius of the hyperplane, or outside it, serve as the support vectors for the model, contributing error penalties that the model seeks to minimize over the fitting iterations.
- DTR: built by splitting the data based on an input feature and threshold value at each level. The model chooses to split the data points on the feature that makes the output values within each child subset as similar as possible. This selection process is repeated to build out the tree. The leaves of the tree represent the prediction regression output values and each is an average of the output values for the data points contained within that leaf.
- RFR: consists of many decision trees that are each constructed using random subsets of the overall dataset and a random subset of features as well. The output from the forest of decision trees is then averaged to get a final output prediction.
- GDB: builds a sequence of decision trees, with each subsequent tree decreasing the residual prediction errors left by the previously built trees. The predictions from the trees are combined sequentially to produce the final regression output.
- KNN: makes a prediction by averaging the output values of the k-nearest training data points to the given input features.

Boosted Data

The next phase involved expanding the available data by taking the ACI dataset and using Regressor 1 to estimate its strength. The ACI dataset has the same seven input features as the original 425 data points, but no associated strength output features. It is assumed here that Regressor 1's estimation of these strength outputs represents the real, or close to real, values. The 9,575 ACI samples were then concatenated with the 425 original samples, created a boosted, 10,000 sample dataset. Note that all of these data points were scaled into the Regressor 1 scaled domain.

Super-Boosted Data

Next, a Generative Adversarial Network was used to generate another 40,000 synthetic datapoints. In particular, the latest version of the ctgan library was used to train the GAN model using all of the input and output features of the Boosted data. These 40,000 samples did not, however, generate with their corresponding output features. Once again, Regressor 1 was used to generate these, with the assumption being made that these are the "real" or near real values. Principal Component Analysis was used to ensure this data fell within roughly the same domain as the real data. Once this was verified, these synthetic samples were then concatenated onto the 10,000 samples of the Boosted data to create the "Super-Boosted" data set. Once again, all data is in the scaled domain from Regressor 1.

Combinatory Investigation

A combinatory investigation was then executed using GDB with RTT of 0.1 and 5 RRS and the super-boosted dataset generated in the prior step. In total there are 127 combinations to test, each to be tested for the 1 RTT across 5 RRS giving 635 total trained models. The mean squared errors across the five RRS were averaged for each combination, and the best combinations were evaluated to inform the training of Regressor 2.

Regressor 2:

A new GDB model was trained, this time on the 50,000 super-boosted data points (all used as training samples) and only with the feature combination identified in the prior step. This model is referred to as "Regressor 2."

Optimization:

Finally, objective and constraint functions were generated to be used in an optimization using the `scipy.optimize.minimize` model. The objective was to maximize 28-Day Compressive Strength ratio'd to mix weight. There was a constraint on total mix weight, as well as the water to cement ratio. Regressor 2 was used to calculate the compressive strength for the objective function. A random initial input concrete mixture was generated as a starting point.

RESULTS AND DISCUSSION

Table 1 shows the features that remain after dropping Age as a feature. The original dataset was filtered to only include data points that are based on 28-day strength, leaving only 425 samples.

Table 1. Example Entries of Original Dataset

Cement	Blast Furnace Slag	Fly Ash)	Water	Superplasticizer	Coarse Aggregate	Fine Aggregate	Concrete compressive strength
540	0	0	162	2.5	1040	676	79.99
540	0	0	162	2.5	1055	676	61.89
380	95	0	228	0	932	594	36.45
266	114	0	228	0	932	670	45.85
475	0	0	228	0	932	594	39.29

Table 2 below shows the testing and training MSEs for all 6 models for each of the 10 RRSs at an RTT of 0.05 and an RTT of 0.1, using the initial dataset outlined in Table 1. Inspecting the table reveals that the gradient boosting regressor and the random forest regressor have the lowest errors. The decision tree regressor appeared to overfit the training data, evident by a zero training error but higher testing error. The k-nearest neighbor regressor and the linear regressor had the highest average error when combining the averages of both RTTs and all the RRSs

Table 2. Training and Testing MSE Values for All 6 Models for Each RTT-RRS

Model	RTT	RRS	MSE Training	MSE Testing
DTR	0.05	1	3.95E-05	0.003799
DTR	0.05	2	3.95E-05	0.010464
DTR	0.05	3	3.95E-05	0.003811
DTR	0.05	4	3.95E-05	0.004999
DTR	0.05	5	3.95E-05	0.002191
DTR	0.05	6	3.95E-05	0.016766
DTR	0.05	7	3.95E-05	0.003469
DTR	0.05	8	3.95E-05	0.007314
DTR	0.05	9	3.67E-05	0.004029
DTR	0.05	10	3.95E-05	0.003974
DTR	0.1	1	4.31E-05	0.012333
DTR	0.1	2	4.37E-05	0.019538
DTR	0.1	3	4.17E-05	0.020351
DTR	0.1	4	4.17E-05	0.007905
DTR	0.1	5	4.52E-05	0.017883
DTR	0.1	6	4.16E-05	0.011148
DTR	0.1	7	4.17E-05	0.007373

DTR	0.1	8	4.17E-05	0.007485
DTR	0.1	9	4.17E-05	0.006478
DTR	0.1	10	4.17E-05	0.008257
GDB	0.05	1	0.001175	0.004611
GDB	0.05	2	0.001159	0.006749
GDB	0.05	3	0.001186	0.004123
GDB	0.05	4	0.001154	0.002817
GDB	0.05	5	0.001273	0.003163
GDB	0.05	6	0.001149	0.004441
GDB	0.05	7	0.001112	0.005528
GDB	0.05	8	0.001138	0.002776
GDB	0.05	9	0.001098	0.005453
GDB	0.05	10	0.001041	0.010441
GDB	0.1	1	0.001221	0.007306
GDB	0.1	2	0.001165	0.008338
GDB	0.1	3	0.001146	0.01002
GDB	0.1	4	0.001207	0.004898
GDB	0.1	5	0.001275	0.006852
GDB	0.1	6	0.001141	0.007431
GDB	0.1	7	0.00119	0.002903
GDB	0.1	8	0.001164	0.004084
GDB	0.1	9	0.001221	0.004011
GDB	0.1	10	0.001265	0.003516
KNN	0.05	1	0.006482	0.008764
KNN	0.05	2	0.006273	0.007704
KNN	0.05	3	0.006339	0.00866
KNN	0.05	4	0.006407	0.012474
KNN	0.05	5	0.006375	0.010018
KNN	0.05	6	0.00648	0.011022
KNN	0.05	7	0.006514	0.009446
KNN	0.05	8	0.006413	0.005436
KNN	0.05	9	0.006215	0.012975
KNN	0.05	10	0.006009	0.011167
KNN	0.1	1	0.006821	0.010848
KNN	0.1	2	0.006527	0.014212
KNN	0.1	3	0.006134	0.014578
KNN	0.1	4	0.006964	0.010951
KNN	0.1	5	0.006985	0.013379
KNN	0.1	6	0.006404	0.012274
KNN	0.1	7	0.00667	0.009818
KNN	0.1	8	0.006623	0.005681
KNN	0.1	9	0.006481	0.00864
KNN	0.1	10	0.006333	0.011301

LNR	0.05	1	0.008918	0.014993
LNR	0.05	2	0.009315	0.007432
LNR	0.05	3	0.009464	0.00452
LNR	0.05	4	0.009285	0.00802
LNR	0.05	5	0.009331	0.007192
LNR	0.05	6	0.009013	0.013234
LNR	0.05	7	0.009228	0.009163
LNR	0.05	8	0.009242	0.008752
LNR	0.05	9	0.009304	0.007641
LNR	0.05	10	0.008817	0.017062
LNR	0.1	1	0.009315	0.011886
LNR	0.1	2	0.00922	0.014121
LNR	0.1	3	0.008545	0.015889
LNR	0.1	4	0.008993	0.011391
LNR	0.1	5	0.009902	0.011085
LNR	0.1	6	0.009187	0.009491
LNR	0.1	7	0.00957	0.006046
LNR	0.1	8	0.009428	0.007513
LNR	0.1	9	0.009375	0.007786
LNR	0.1	10	0.009249	0.008973
RFR	0.05	1	0.000893	0.005045
RFR	0.05	2	0.000926	0.006202
RFR	0.05	3	0.000913	0.003484
RFR	0.05	4	0.000856	0.004415
RFR	0.05	5	0.000931	0.002604
RFR	0.05	6	0.000872	0.004686
RFR	0.05	7	0.000896	0.002612
RFR	0.05	8	0.000908	0.00369
RFR	0.05	9	0.000928	0.003318
RFR	0.05	10	0.000801	0.008103
RFR	0.1	1	0.000879	0.007068
RFR	0.1	2	0.000891	0.009535
RFR	0.1	3	0.000858	0.00973
RFR	0.1	4	0.000986	0.005944
RFR	0.1	5	0.000971	0.008857
RFR	0.1	6	0.000848	0.008206
RFR	0.1	7	0.000878	0.003792
RFR	0.1	8	0.000912	0.004727
RFR	0.1	9	0.000942	0.005054
RFR	0.1	10	0.000948	0.004163
SVR	0.05	1	0.005273	0.008918
SVR	0.05	2	0.005294	0.01103
SVR	0.05	3	0.00542	0.00692

SVR	0.05	4	0.005469	0.008119
SVR	0.05	5	0.005292	0.006867
SVR	0.05	6	0.005361	0.007088
SVR	0.05	7	0.005426	0.006124
SVR	0.05	8	0.005394	0.005052
SVR	0.05	9	0.005321	0.006534
SVR	0.05	10	0.005161	0.015611
SVR	0.1	1	0.005517	0.010063
SVR	0.1	2	0.005365	0.011875
SVR	0.1	3	0.005086	0.011056
SVR	0.1	4	0.005313	0.006461
SVR	0.1	5	0.005643	0.008695
SVR	0.1	6	0.005503	0.007701
SVR	0.1	7	0.005432	0.005661
SVR	0.1	8	0.005583	0.005501
SVR	0.1	9	0.005683	0.007553
SVR	0.1	10	0.005303	0.007103

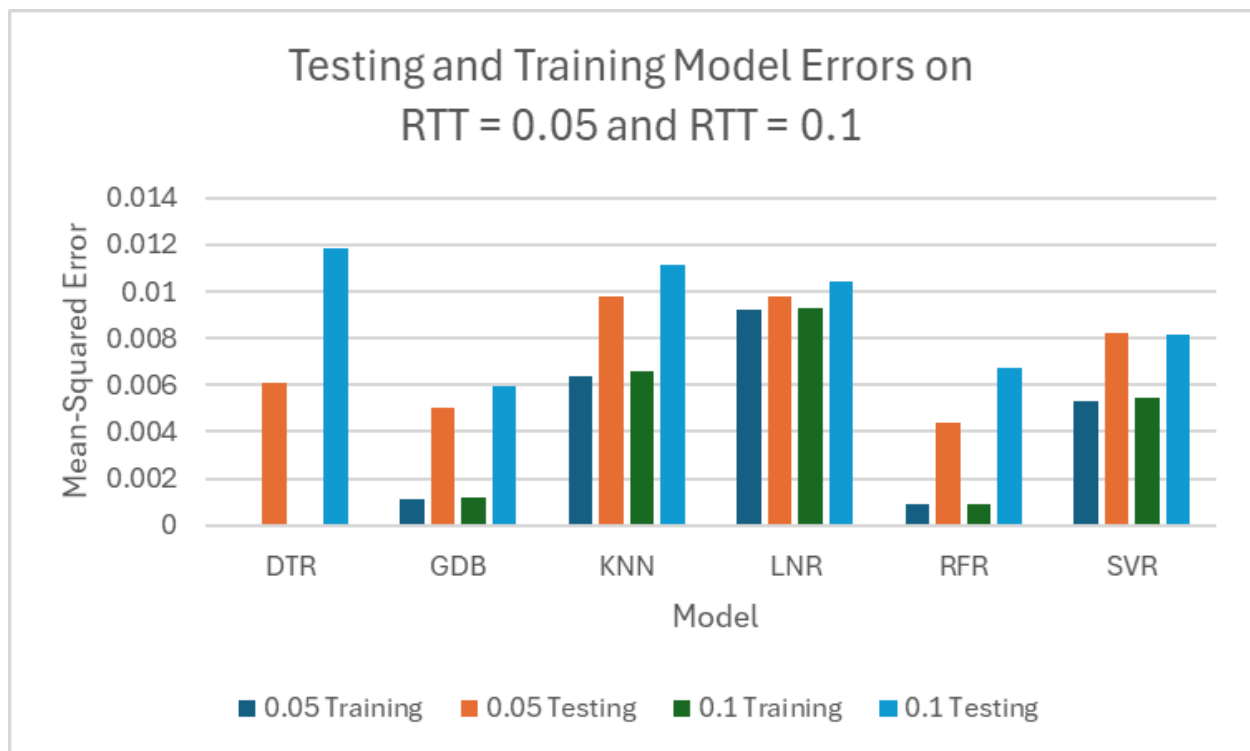


Figure 1. Testing and training error for various models at RTT = 0.05 and RTT = 0.1

Figure 1 shows the resulting testing and training errors for each of the 6 models after being fit on the original dataset consisting of 425 samples. From the chart, the gradient boosting model had the lowest average testing error for the two RTTs. The random forest regressor had lower

training error than the gradient boosting model, but a higher average testing error. Therefore, the best model to move forward with was chosen to be the gradient boosting model. Representative real vs estimated plots for the GBR trained models are shown below in Figures 2 and 3, with Figure 2 corresponding to a model with $RTT = 0.05$ and Figure 3 to one with $RTT = 0.1$.



Figure 2: Real vs. Estimated results for Training (left) and Testing (right) of the GBR model for $RTT = 0.05$ and $RRS = 10$.

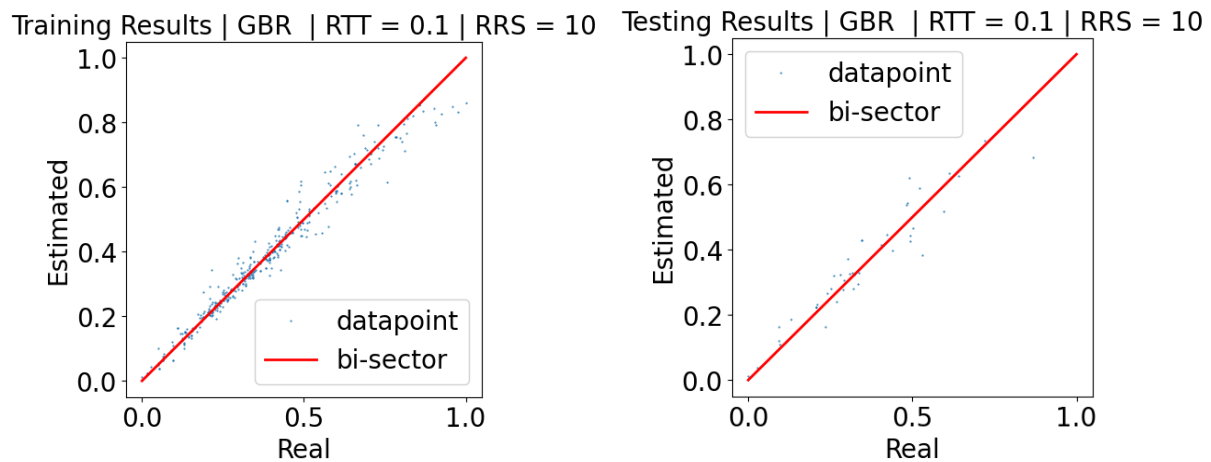


Figure 3: Real vs. Estimated results for Training (left) and Testing (right) of the GBR model for $RTT = 0.1$ and $RRS = 10$.

Once GDB was identified as the regression model of choice and Regressor 1 was trained, the plot shown below in Figure 4 was generated.

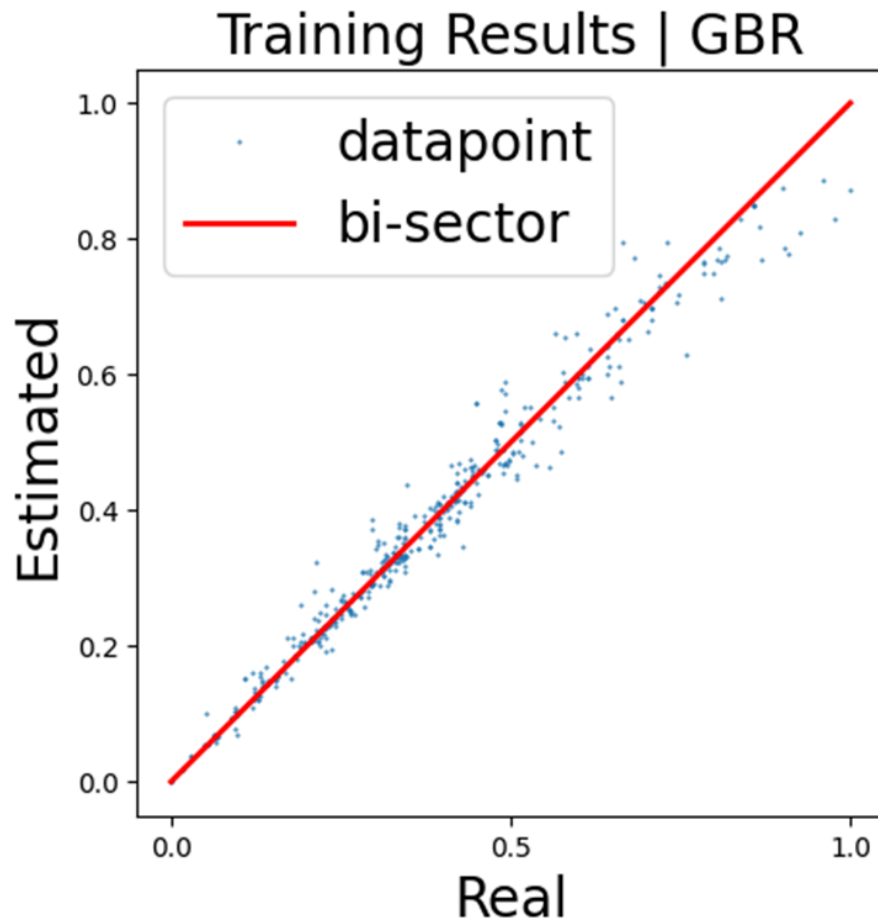


Figure 4. Training Real versus Estimated Compressive Strength Values for Regressor 1

Figure 4 shows the real versus estimated concrete strength values from training the gradient boosting regressor on all 425 data points from the original dataset. The red bi-sector line represents a perfect 1:1 correlation. The closer the points are to the bi-sector, the more accurately the model predicted them. The model appears to predict more accurately at the lower end of the scaled compressive strengths. This same behavior appears to be reflected in Figures 2 and 3, a good sign as they are using the same method.

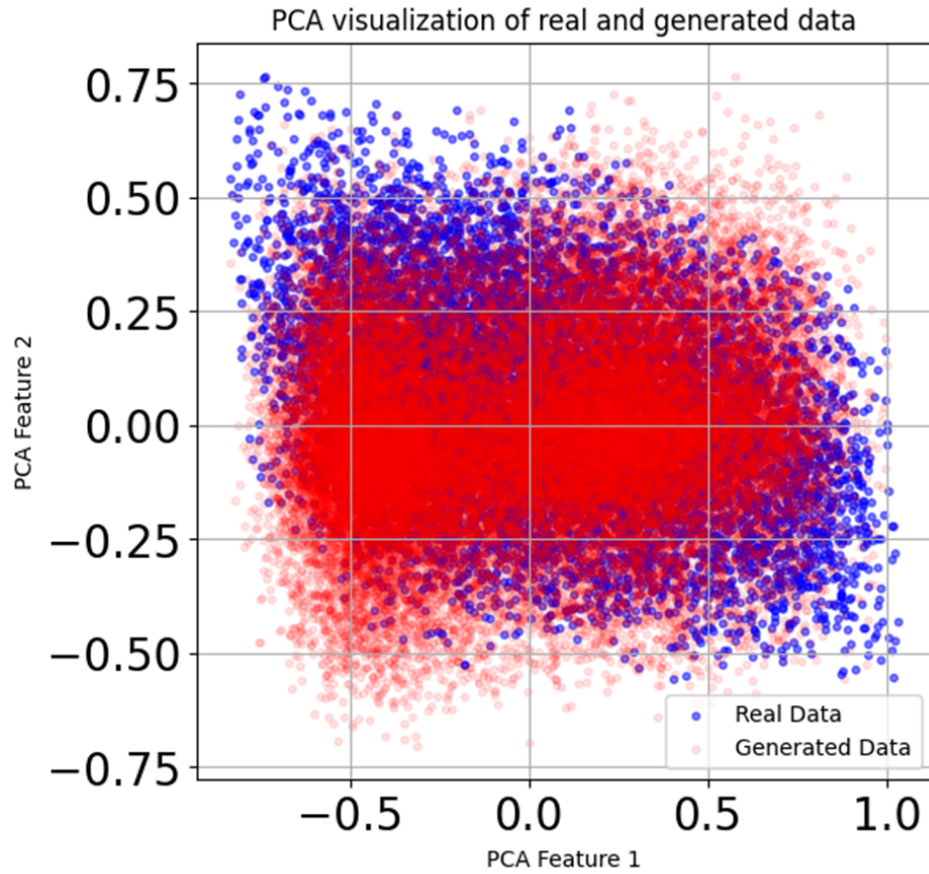


Figure 5. Principal Component Analysis of Real and Generated Data

Figure 5 shows the real data from the original 10,000 points in blue, and the 40,000 GAN generated data points in red. The visualization serves the purpose of validating that the generated data reflects the patterns in the original data. Based on the figure, the generated data seems to match the real data for the most part. The centers of both distributions appear to align nicely. However, there is an apparent absence of generated data points when feature 1 is low and feature 2 is high, and also where feature 1 is high and feature 2 is low. This indicates that the generated data may not exactly mirror the distribution of the original dataset.

Table 3. Gradient Boosted Regressor Best Feature Combination for Testing and Training

	Cement	Blast Furnace Slag	Fly Ash	Water	Superplast icizer	Coarse Aggregate	Fine Aggregate	MSE Training	MSE Testing
GDB Training	1	1	1	1	1	1	1	0.000139	0.000169
GDB Testing	1	1	1	1	1	1	1	0.000139	0.000169

Table 3 shows the feature combinations that resulted in the lowest MSEs for training and testing. The table shows that the best combination consisted of all the features for both training and testing, hence why the MSE training and testing values match for both rows.

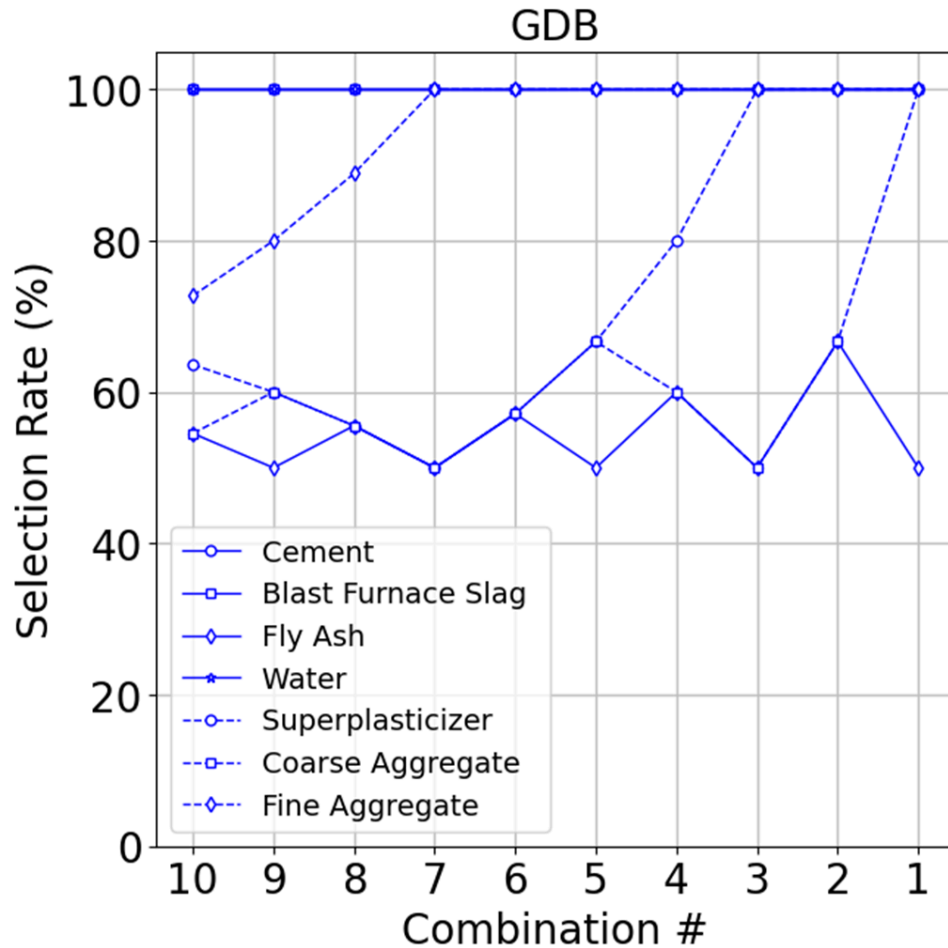


Figure 6. Cumulative feature selection rates among the top performing combinations

The figure above shows the feature selection rates for the combinations with the lowest MSEs. Of all the features, Fly Ash had the lowest selection rate among the top combinations, only being selected half of the time. Coarse aggregate was also not selected consistently, only going up to 100% for the final combination. As neither appear to be critical drivers, both were neglected in the training of Regressor 2 on the 50,000 sample super-boosted dataset.



Figure 7. Training Real versus Estimated Compressive Strength Values for Regressor 2

Figure 7 shows the real versus estimated compressive strength values for regressor 2. Datapoints are shown for each of the 50,000 data points the gradient boosted regressor was trained on. A data point on the red bi-sector represents an accurate prediction. Points further away from the bi-sector represent a higher prediction error. From the figure, the model produced quite accurate estimations as the data points closely follow the bi-sector for the full range of real, scaled strengths.

Table 4: Feature Values from Predicted Maximal Compressive Strength / Weight Ratio

	Cement	Blast Furnace Slag	Fly Ash	Water	Superplasticiz er	Coarse Aggregate	Fine Aggregate	Total
Scaled Value	0.497	0.586	0.236	0.252	0.799	0.217	0.185	
Actual Value	319.423	210.763	47.189	153.324	25.719	875.687	667.900	2300.00 kg/m ³
Strength								72.344

Table 4 shows the scaled and unscaled feature values representing the mixture the optimizer found to have the highest compressive strength/weight ratio. The optimizer was subject to the constraints of the weight being between 2300 and 2500 kg/m³ and the water/cement ratio being between 0.48 and 0.59. The optimizer identified the best mixture to weigh almost exactly 2300 g/m³ and have a compressive strength of 72.344 MPa. These values resulted in a strength-to-weight ratio of 0.03145 MPa * m³/kg.

Question 1) How can we determine if our optimization results would have been more accurate/reliable if we had just used the 425 initial data points (instead of super-boosted data points) in steps 7, 8, and 9?

To determine if the optimization results would have been more reliable with only the 425 initial data points instead of using the super-boosted data points, the model trained only on the 425 data points and the super-boosted trained model's performance on a separate dataset could be compared. A separate, external dataset featuring the same features and outputs would provide a level testing benchmark for each of the models. Accuracies and errors for each model could be compared. If the model trained on only the 425 data points showed higher accuracy, there is a good chance that it would predict the highest strength-to-weight ratio more reliably than the model trained on the super-boosted data. In the absence of an additional dataset to experiment on, the optimized results from each model could be visualized against the original 425 data points as an accuracy and reliability test. To compare fairly, the optimization process would need to be run in the same manner for both models. Since the optimization was executed using an array of random initial conditions, both models would need to use the same set of initial conditions, or a seed could be applied for reproducible results. After the optimization, the compressive strength-to-weight ratios for each model's optimized mixes could be plotted against the ratios for the original 425 data points. This comparison could help determine whether the optimized solutions are consistent with the trends present in the original, experimental dataset.

Question 2) Which Regressor (1 or 2) is generally more accurate and reliable in estimating Concrete Compressive Strength? Why?

Since Regressor 1 was only trained on 425 data samples and Regressor 2 was trained on 50,000 samples, Regressor 2 should generally be more accurate and reliable in estimating concrete compressive strength. Having vastly more data points to train on should allow Regressor 2 to more densely learn the patterns of the input features and how they correlate to the compressive strength of the concrete mixture. With only 425 data points, the input space is sparsely populated compared to having 50,000 points. The additional points allow the model to fill in the gaps left between the 425 data points. However, because the additional data points were generated from a model that was trained on the original dataset, they largely encode and expand on the information learned by Regressor 1. Ultimately, Regressor 2 may provide more consistent predictions across the feature space, but the estimates may not perfectly represent actual physical mixture compressive strengths in the real world.

Question 3) In our GAN model, we relied on subjective PCA plot observation to see if the generated and boosted data are scattered uniformly or homogeneously. Look online and list at least three quantitative measurements that can be used to evaluate the quality of the generated data points in step 6. For each measurement, provide a minimum 3 line explanations.

- **Wasserstein Distance:** The Wasserstein distance measures the cost required to change a point from one dataset into a point in the other dataset. The distance a point has to be moved contributes to the cost. That means datasets that are dissimilar will require points to be transported further, increasing the overall cost and Wasserstein distance value. In this case, it provides a quantitative value representing how different the original dataset is from the generated boosted data. A smaller Wasserstein distance indicates that the generated data more closely matches the distribution of the original data.
- **Jensen-Shannon Divergence:** The Jensen-Shannon divergence is another way to quantitatively compare two datasets. Jensen-Shannon divergence is found by first creating an average distribution between the two datasets. Then, how different each dataset is from the average is calculated. After determining how different both datasets are from the averaged dataset, those values are averaged together. The divergence results in an output between 0 and 1, where 0 means the distributions are exactly the same, and 1 means they are completely different. Therefore, the lower the Jensen-Shannon divergence, the more similar the datasets are.
- **Distance to Closest Record:** Instead of looking at how similar datasets or distributions are as a whole, distance to closest record checks how close each generated sample is to the nearest real sample. The purpose of using distance to closest record is to determine if the generated data is too much of a copy of the original data. For quality data, the distance to closest record values should not be tiny, but they should also not be too large either. Miniscule values can indicate copied original data, while large values can represent inaccurate generated data.

Question 4) Would it have been more accurate/reliable if we had first identified the best combinations of features in step 2 and followed the other steps? Why?

If the best combinations of features were identified in step 2, the overall experiment may have been more accurate or reliable. Identifying the best combinations of features serves the purpose of eliminating any features that may be adding unnecessary information or noise that does not contribute to the patterns of the overall dataset. Using a specific combination of features can improve the accuracy and reliability of a model as long as the features that were left out do not contain any valuable information. In this experiment, using the best combination of features could have improved the initial model accuracy, which would in turn create more accurate and reliable output values for the ACI data, making the generated GAN data points more accurate and reliable as well. That improvement would flow over into training Regressor 2, and thus could lead to more accurate and reliable optimization results.

Question 5) If you were to redo step 6 with 5 RTTs and 100 RRSs and for all the seven models of step 2, how would you distribute your computation load on multiple nodes and cores on MARCC?

If the training were redone with 5 RTTs and 100 RRSs for all the models from step 2, the computation load could be distributed by model and RTT combination pairs. 7 models and 5 RTTs would result in 35 independent jobs, with each job responsible for running the 100 RRSs for its respective model and RTT pair. On the shared partition, the Rockfish cluster can have up to 48 cores per node, so either 25 or 34 processors per node could be used to execute the RRSs. The overall computation load distribution would be 35 nodes, each with its own job consisting of a model and RTT pair, with 25 or 34 processors used on each node to execute the 100 RRSs in parallel.

Question 6) If you were to benefit from one TensorFlow model similar to the ones we observed in Module 4, in which step would you develop and use it?

If the experiment were to benefit from a deep neural network similar to what was used in Module 4, it could be developed in step 8 as the second regressor. Since deep neural networks generally perform better with larger amounts of data to train on compared to simpler models, it would make more sense to train the model after the super boosted data points were generated. If a deep neural network were trained on the original 425 data points, there would be a greater chance of the model overfitting the training data depending on the number of layers, dropout, and other parameters. Regardless, if the neural network was developed as regressor 2, it would still be beneficial to compare its performance metrics against the best performing model from the previous steps in the experiment.

CONCLUSION

This experiment evaluated multiple regression models for predicting concrete compressive strength using an initial dataset of 425 samples. The models were compared using MSE across two different RTTs and 10 RRSs, and the best performing model was selected for the later stages of the experiment. Additional compressive strength outputs were generated for the ACI Based Mix dataset to expand the dataset to 10,000 samples. A generative adversarial network was then used to create 40,000 additional samples, with the generated data compared against the existing data using a PCA graph to verify the distributions remained similar. After building the 50,000 point, super-boosted dataset, the selected model was retrained and used in a feature combinatory analysis before being incorporated into an optimization routine to maximize the concrete compressive strength-to-weight ratio subject to constraints on the mixture weight and water-to-cement ratio. The mixture weight was to remain between 2,300 and 2,500 kg/m³, and the water-to-cement ratio to remain between 0.48 and 0.59.

Of the initial models, gradient boosting produced the strongest overall regression performance and was chosen to be trained on the overall set of 50,000 points. The feature combinatory analysis showed that the best performing combination included all 7 input features, although fly ash and coarse aggregate appeared less frequently among the highest-ranked combinations

compared to the other inputs. During optimization, the gradient boosting model proved sensitive to initial conditions. Many optimization runs terminated after relatively few iterations. To combat that, numerous random starting conditions were used to increase the optimizer's chance of finding a good solution. Ultimately, the experiment demonstrated how regression modeling, synthetic data generation, feature analysis, and constrained optimization can be combined to identify a concrete mixture with a high strength-to-weight ratio.

Pros

- The experiment featured a multi-paradigm approach to show how training and evaluating various machine learning models can lead to more accurate results. Even if one model seems like it would be best fit for a problem, doing an early training run to identify the best performer can produce large gains when scaled to larger datasets.
- Leveraging multiple RTTs and RRSs to select the best model from the initial training ensured that the model that performed the best consistently was chosen for the rest of the experiment, even if one model produced particularly promising results on a single RTT value.
- Using additional, unlabeled input data samples and predicting their outputs, along with a generative adversarial network, provided a way to increase the size of the training dataset. This simulated how extra data points may be generated in industry in order to effectively train a model.
- Plotting the real versus estimated training results at multiple points throughout the model development process provided useful feedback in how the model was performing. Even though a large portion of the training data was generated synthetically, seeing how effectively the model predicted the training outputs was still necessary in ensuring the accuracy of the model.
- Including the combinatory training runs was a valuable step in developing the model ultimately used to optimize the concrete mixture. Using a different combination of features to train the same model can result in varying prediction accuracies and errors. Making sure the optimal combination of features is used to train the model the final time adds a layer of protection against noisy data and overfitting.

Cons

- The experiment involved utilizing data from a generative adversarial network. While using the GAN increased the size of the training dataset, 9,575 or the first 10,000 data points had outputs generated by another model instead of them being labeled already. Any error in the model's predicted outputs might be reflected in the data generated by the GAN.
- Since the GAN was partially trained on predicted data, adding another method besides visually checking a principal component analysis might be useful. The PCA graph was only two-dimensional, so the generated data may have varied from the original training data in a way that was not visible on the graph.
- In the experiment, the best model was selected based on which had the lowest MSE after being trained on the original dataset. This included all of the features. Only after selecting the best model was the feature combinatory analysis ran. If the combinatory

analysis was executed for all of the models, another model may have performed better after only seeing the original dataset.

- A gradient boosted machine learning model creates decision trees in order to estimate output values. Decision trees do not provide a continuous output surface, so the optimizer ran into issues when trying to traverse the prediction surface. This was evident after tracking the number of iterations the optimizer went through when finding the best mixture.
- Even though selecting the first regressor examined the performance of 6 different models, none of the hyperparameters for the models were set or tuned. Each model has its own set of hyperparameters that can greatly affect the performance of the model. Tuning the hyperparameters may have resulted in an alternate model being selected for the remainder of the experiment.

Changes to Experiment

If we were to conduct this experiment again, the first change to implement would be running the feature combination analysis on each of the 6 starting models. Although this would greatly increase the computational load of the initial training, it would be a worthwhile addition as the performance outcomes would affect the rest of the experiment going forward. Through the combinatorial analysis, the best version of each model could be directly compared. From that point, we would carry the top two performing models into the next steps.

Both selected models and their optimal feature combinations would be used to predict the output values for the ACI Based Mix dataset. These output values could then be averaged to arrive at the final strength estimates to go with the provided input features. Averaging the output values of both well performing models reduces dependence on a single model and could improve the quality of the generated data. In addition to averaging the output values, a quantitative method could be used to verify that the generated output samples match the output distribution of the samples from the original 425 data points.

Next, both models would again be trained, this time using the concatenated dataset of 10,000 points. The generative adversarial network would be used to synthesize 40,000 new data points before being verified through the PCA graph and another quantitative method such as Jensen-Shannon Divergence to evaluate similarity in the distributions. Following validation of the generated data, both models would be fit to the total dataset to prepare for the optimization. Fitting iterations with modified hyperparameters on each model would be useful at this stage as well, with progress being measured by training error and real versus estimated plots.

Finally, after the hyperparameters for both models are tuned, the optimization function and constraints would be implemented. Since the optimizer can be very sensitive to initial conditions, a loop of many random starting conditions or seeds could be used to improve the chances of the optimizer finding the global maximum. If the selected methods were not continuous, perhaps another optimizer could be implemented or techniques to escape local minima could be added. The optimizer results for both models could be compared, examining

number of iterations, overall mix weight, water-to-cement ratio, and, of course the objective strength-to-weight ratio.

Member Contributions:

Graeme Marshall:

- Regressor and optimizer implementation
- Report and process writing

Logan Griffin:

- Data processing, regressor, and GAN implementation
- Report and process writing

References

American Concrete Institute. (n.d.). *Standards for 7-day and 28-day strength test results*.

American Concrete Institute.

<https://www.concrete.org/frequentlyaskedquestions/faqid/13.aspx>

Evidently AI. (2022, June 20). *Which test is the best? We compared 5 methods to detect data drift on large datasets*. <https://www.evidentlyai.com/blog/data-drift-detection-large-datasets>

The Synthetic Mind. (2025, December 26). *Wasserstein Distance & Optimal Transport — Fully Explained* [Video]. YouTube. https://www.youtube.com/watch?v=88ONbF_b3VE

Yeh, I. (1998). Concrete Compressive Strength [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5PK67>.